word2vec

How a computer learns what words mean, from nothing but text

Based on Mikolov et al., 2013

Today's plan

1. **The problem**: why computers struggle with word meaning
2. **The core idea**: learn meaning by predicting neighbours
3. **How it learns**: negative sampling and hierarchical softmax
4. **What you get**: similarity, analogies, and the limits

PART 1

The problem

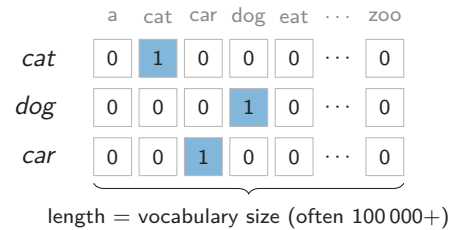
To a computer, a word is just a symbol

Machine learning models work with **numbers**. So how do we turn words into numbers?

To a computer, a word is just a symbol

Machine learning models work with **numbers**. So how do we turn words into numbers?

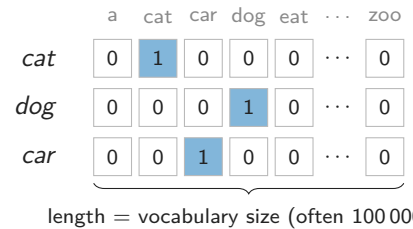
The simplest idea: **one-hot vectors**. One slot per word in the vocabulary.



To a computer, a word is just a symbol

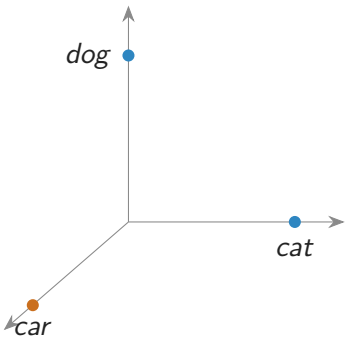
Machine learning models work with **numbers**. So how do we turn words into numbers?

The simplest idea: **one-hot vectors**. One slot per word in the vocabulary.

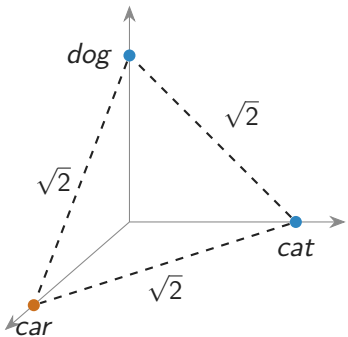


Problem: these numbers say nothing about what the words *mean*.

One-hot: every word is equally far from every other

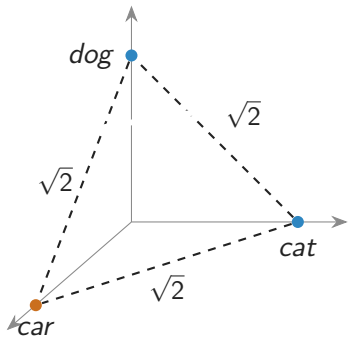


One-hot: every word is equally far from every other



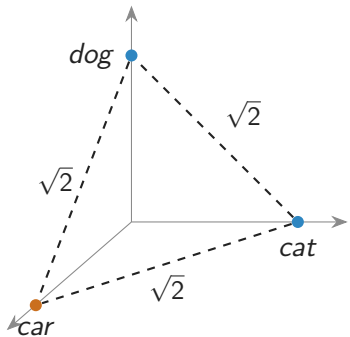
- *cat* is exactly as far from *dog* as from *car*.

One-hot: every word is equally far from every other



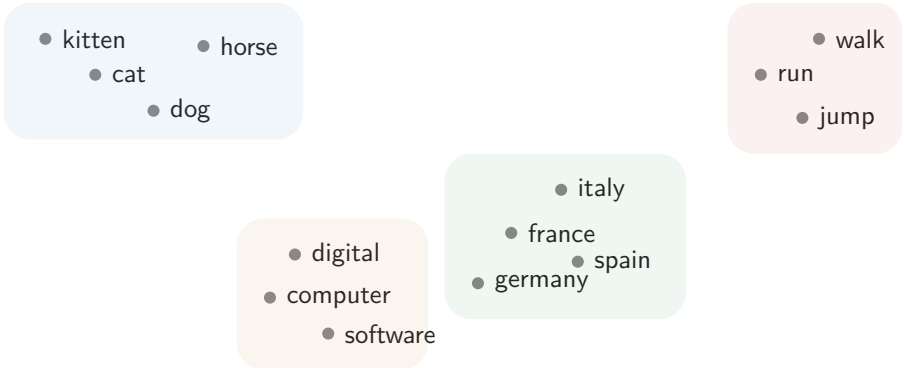
- *cat* is exactly as far from *dog* as from *car*.
- Anything a model learns about *cats* tells it **nothing** about *dogs*.

One-hot: every word is equally far from every other

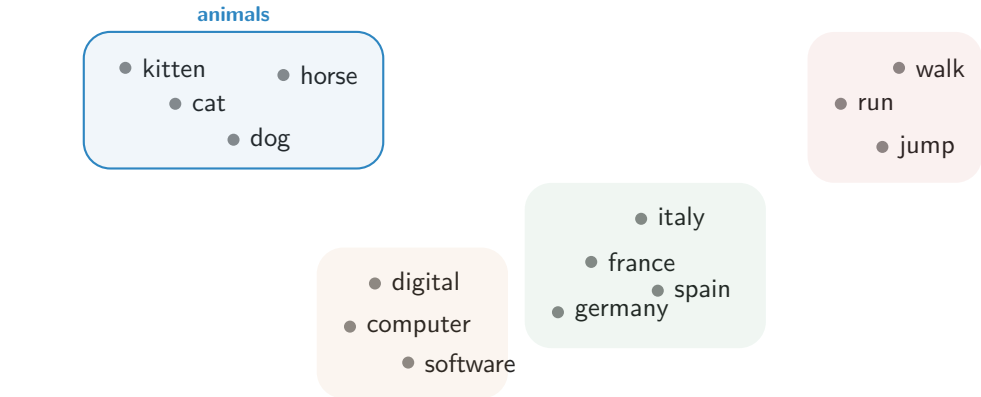


- *cat* is exactly as far from *dog* as from *car*.
- Anything a model learns about *cats* tells it **nothing** about *dogs*.
- And the vectors are huge and almost all zeros.

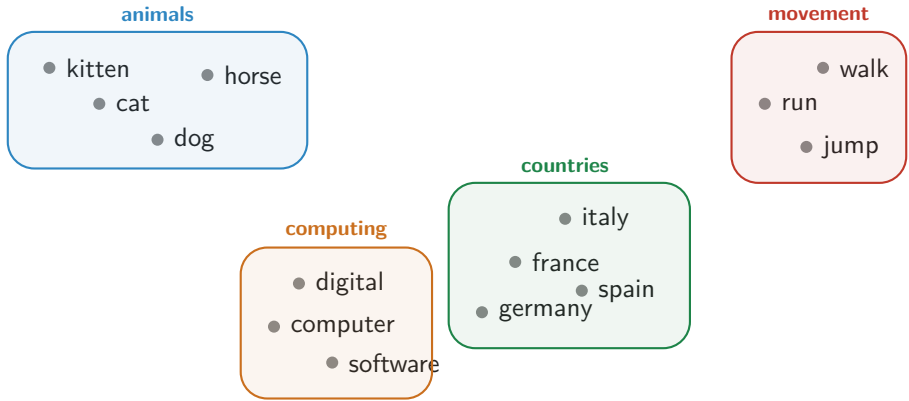
What we want instead: a map of meaning



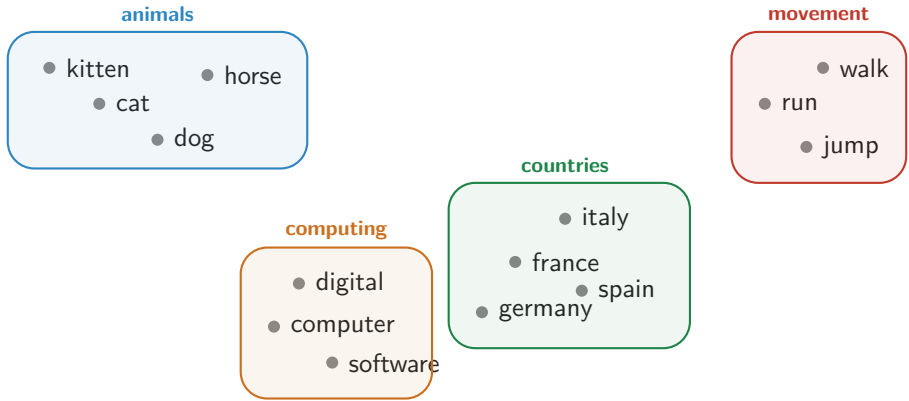
What we want instead: a map of meaning



What we want instead: a map of meaning

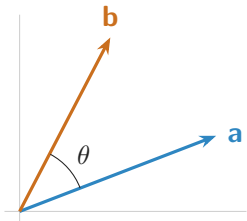


What we want instead: a map of meaning



Each word becomes a **short list of numbers** (say 200): a point in space.
Nearby points = similar meanings. (We can only draw 2 dimensions.)

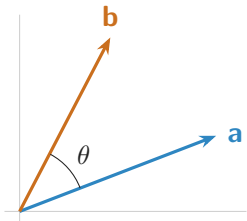
Quick refresher: vectors and similarity



Dot product: multiply matching entries, add them up

$$\mathbf{a} \cdot \mathbf{b} = a_1 b_1 + a_2 b_2 + \cdots + a_d b_d$$

Quick refresher: vectors and similarity



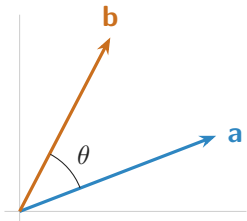
Dot product: multiply matching entries, add them up

$$\mathbf{a} \cdot \mathbf{b} = a_1 b_1 + a_2 b_2 + \cdots + a_d b_d$$

Cosine similarity: the dot product of the directions only

$$\cos\theta = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \, \|\mathbf{b}\|}$$

Quick refresher: vectors and similarity



Dot product: multiply matching entries, add them up

$$\mathbf{a} \cdot \mathbf{b} = a_1 b_1 + a_2 b_2 + \cdots + a_d b_d$$

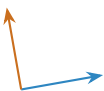
Cosine similarity: the dot product of the directions only

$$\cos\theta = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \, \|\mathbf{b}\|}$$



$\cos\theta \approx 1$

same direction



$\cos\theta \approx 0$

unrelated



$\cos\theta \approx -1$

opposite

Where could meaning come from?

"You shall know a word by the company it keeps."

J. R. Firth, linguist, 1957

Where could meaning come from?

"You shall know a word by the company it keeps."

J. R. Firth, linguist, 1957

Fill in the blank:

I fed my _____ and took it to the vet.

Where could meaning come from?

"You shall know a word by the company it keeps."

J. R. Firth, linguist, 1957

Fill in the blank:

I fed my _____ and took it to the vet.

cat ✓ dog ✓ rabbit ✓ car × idea ×

Where could meaning come from?

"You shall know a word by the company it keeps."

J. R. Firth, linguist, 1957

Fill in the blank:

I fed my _____ and took it to the vet.

cat ✓ dog ✓ rabbit ✓ car × idea ×

Distributional hypothesis: words that appear in similar contexts have similar meanings.

Where could meaning come from?

"You shall know a word by the company it keeps."

J. R. Firth, linguist, 1957

Fill in the blank:

I fed my _____ and took it to the vet.

cat ✓ dog ✓ rabbit ✓ car × idea ×

Distributional hypothesis: words that appear in similar contexts have similar meanings.

So if we capture each word's *typical neighbours*, we capture its meaning.

PART 2

The core idea

The trick: invent a prediction game

1. Take a large amount of ordinary text (e.g. Wikipedia).

The trick: invent a prediction game

1. Take a large amount of ordinary text (e.g. Wikipedia).
2. Play a guessing game: **predict a word from its neighbours** (or the reverse).

The trick: invent a prediction game

1. Take a large amount of ordinary text (e.g. Wikipedia).
2. Play a guessing game: **predict a word from its neighbours** (or the reverse).
3. Give every word a vector, start them **random**, and adjust them to win the game.

The trick: invent a prediction game

1. Take a large amount of ordinary text (e.g. Wikipedia).
2. Play a guessing game: **predict a word from its neighbours** (or the reverse).
3. Give every word a vector, start them **random**, and adjust them to win the game.
4. When training ends, **throw away the game and keep the vectors**.

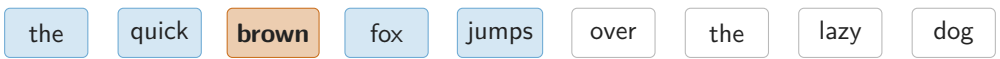
The trick: invent a prediction game

1. Take a large amount of ordinary text (e.g. Wikipedia).
2. Play a guessing game: **predict a word from its neighbours** (or the reverse).
3. Give every word a vector, start them **random**, and adjust them to win the game.
4. When training ends, **throw away the game and keep the vectors**.

No human labels anything. The text provides its own answers.
This is called **self-supervised learning**, and it is how today's language models are trained too.

Turning text into training examples

Slide a **window** over each sentence. The middle word is the **centre**; the words around it are its **context**.

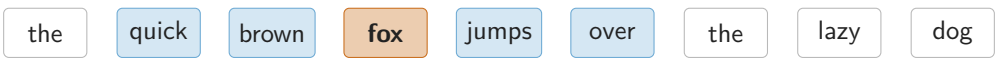


window = 2 words on each side

Pairs: (brown, the) (brown, quick) (brown, fox) (brown, jumps)

Turning text into training examples

Slide a **window** over each sentence. The middle word is the **centre**; the words around it are its **context**.



window = 2 words on each side

Pairs: (fox, quick) (fox, brown) (fox, jumps) (fox, over)

Turning text into training examples

Slide a **window** over each sentence. The middle word is the **centre**; the words around it are its **context**.



window = 2 words on each side

Pairs: (jumps, brown) (jumps, fox) (jumps, over) (jumps, the)

Turning text into training examples

Slide a **window** over each sentence. The middle word is the **centre**; the words around it are its **context**.



window = 2 words on each side

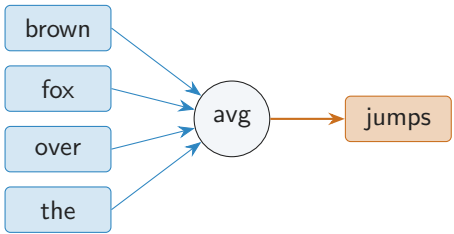
Pairs: (jumps, brown) (jumps, fox) (jumps, over) (jumps, the)

A billion words of text gives **billions** of free (centre, context) examples.

Two ways to play the game

CBOW

"continuous bag of words"

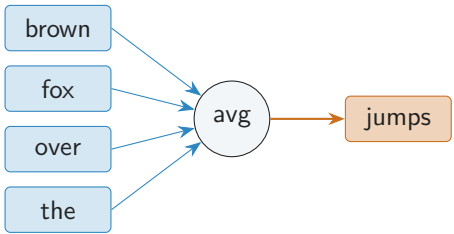


neighbours $\rightarrow$ guess the centre word

Two ways to play the game

CBOW

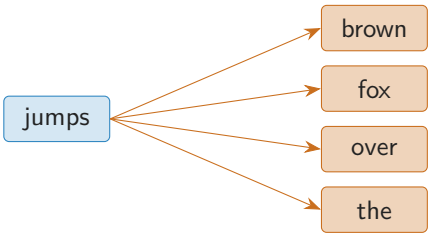
"continuous bag of words"



neighbours → guess the centre word

Skip-gram

"skip around the word"

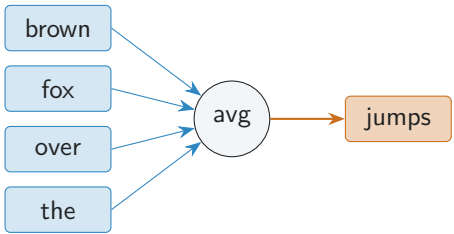


centre word → guess each neighbour

Two ways to play the game

CBOW

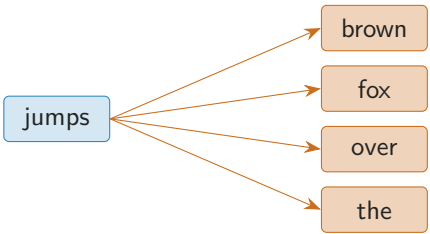
"continuous bag of words"



neighbours → guess the centre word

Skip-gram

"skip around the word"



centre word → guess each neighbour

CBOW is **faster**; skip-gram is slower but **better for rare words**. Both produce good vectors.

The whole model: two vectors per word and a dot product

Every word w gets two vectors:

- $\mathbf{v}_w$: the word as the **input** (kept)
- $\mathbf{u}_w$: the word as a **neighbour**

The whole model: two vectors per word and a dot product

Every word w gets two vectors:

- $\mathbf{v}_w$: the word as the **input** (kept)
- $\mathbf{u}_w$: the word as a **neighbour**

How likely is o to be a neighbour of c?

$$\text{score} = \mathbf{v}_c \cdot \mathbf{u}_o$$

The whole model: two vectors per word and a dot product

Every word w gets two vectors:

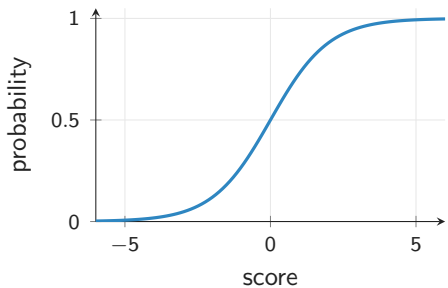
- $\mathbf{v}_w$: the word as the **input** (kept)
- $\mathbf{u}_w$: the word as a **neighbour**

How likely is o to be a neighbour of c?

$$\text{score} = \mathbf{v}_c \cdot \mathbf{u}_o$$

Squash the score into a probability with the **sigmoid**:

$$P = \sigma(\text{score}) = \frac{1}{1 + e^{-\text{score}}}$$



The whole model: two vectors per word and a dot product

Every word w gets two vectors:

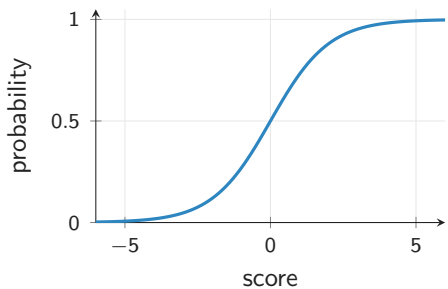
- $\mathbf{v}_w$: the word as the **input** (kept)
- $\mathbf{u}_w$: the word as a **neighbour**

How likely is o to be a neighbour of c?

$$\text{score} = \mathbf{v}_c \cdot \mathbf{u}_o$$

Squash the score into a probability with the **sigmoid**:

$$P = \sigma(\text{score}) = \frac{1}{1 + e^{-\text{score}}}$$



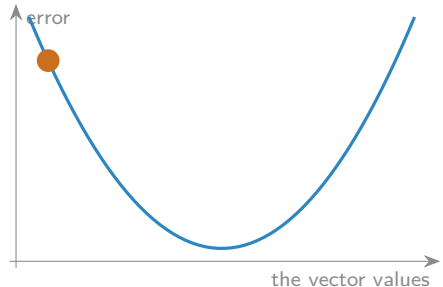
That's all. No deep network: just a dot product, a sigmoid, and lots of data.

PART 3

How it learns

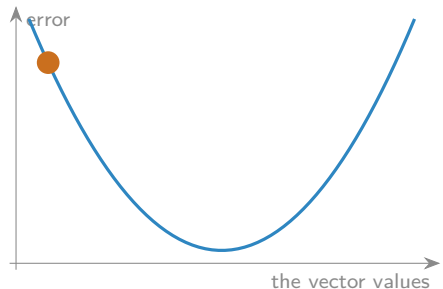
What does “learning” mean?

1. Start with **random** vectors.



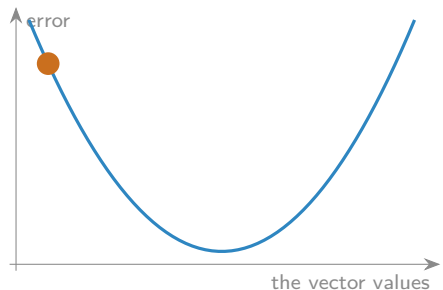
What does “learning” mean?

1. Start with **random** vectors.
2. Show the model one example; it makes a guess.



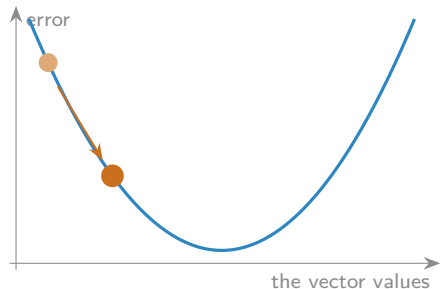
What does “learning” mean?

1. Start with **random** vectors.
2. Show the model one example; it makes a guess.
3. Measure the **error**: how wrong was it?



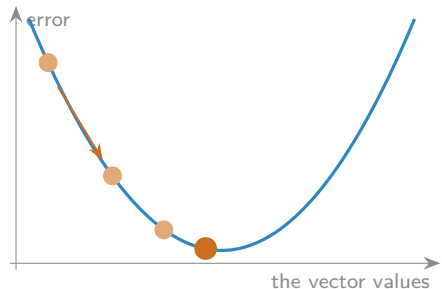
What does “learning” mean?

1. Start with **random** vectors.
2. Show the model one example; it makes a guess.
3. Measure the **error**: how wrong was it?
4. **Nudge** the numbers slightly to reduce the error.



What does “learning” mean?

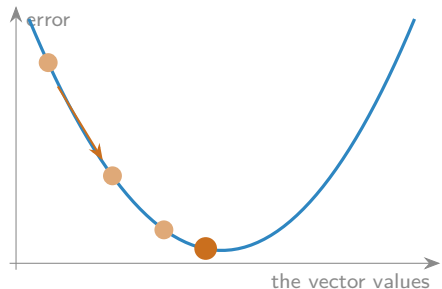
1. Start with **random** vectors.
2. Show the model one example; it makes a guess.
3. Measure the **error**: how wrong was it?
4. **Nudge** the numbers slightly to reduce the error.
5. Repeat, millions of times.



What does “learning” mean?

1. Start with **random** vectors.
2. Show the model one example; it makes a guess.
3. Measure the **error**: how wrong was it?
4. **Nudge** the numbers slightly to reduce the error.
5. Repeat, millions of times.

The size of each nudge is the **learning rate**. It starts large and shrinks as training goes on.



The obvious approach: a probability for every word

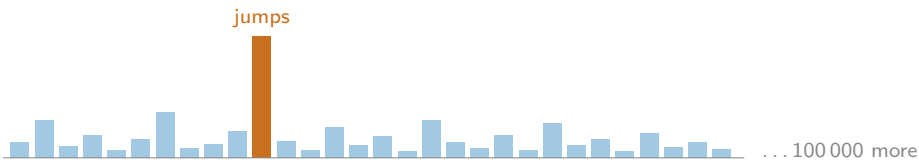
To guess the neighbour of *fox*, score **every** word in the vocabulary and turn the scores into probabilities that add up to 1 (the **softmax**):

$$P(o \mid c) = \frac{e^{\mathbf{v}_c \cdot \mathbf{u}_o}}{\sum_{w \in \text{ all words}} e^{\mathbf{v}_c \cdot \mathbf{u}_w}}$$

The obvious approach: a probability for every word

To guess the neighbour of *fox*, score **every** word in the vocabulary and turn the scores into probabilities that add up to 1 (the **softmax**):

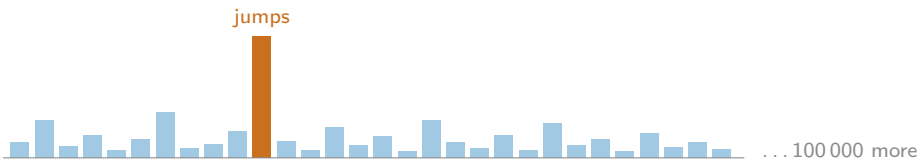
$$P(o \mid c) = \frac{e^{\mathbf{v}_c \cdot \mathbf{u}_o}}{\sum_{w \in \text{all words}} e^{\mathbf{v}_c \cdot \mathbf{u}_w}}$$



The obvious approach: a probability for every word

To guess the neighbour of *fox*, score **every** word in the vocabulary and turn the scores into probabilities that add up to 1 (the **softmax**):

$$P(o \mid c) = \frac{e^{\mathbf{v}_c \cdot \mathbf{u}_o}}{\sum_{w \in \text{ all words}} e^{\mathbf{v}_c \cdot \mathbf{u}_w}}$$



Problem: the bottom of the fraction needs **100 000 dot products** for *every* example, and there are billions of examples. Far too slow.

Fix #1: negative sampling. Ask “real or fake?”

Replace *“which of 100 000 words is the neighbour?”* with a yes/no question: *“is this pair real?”*

(fox, jumps) from the text → **real** (label 1)

Fix #1: negative sampling. Ask “real or fake?”

Replace *“which of 100 000 words is the neighbour?”* with a yes/no question: *“is this pair real?”*

(fox, jumps)

from the text → **real** (label 1)

(fox, piano)

(fox, 1987)

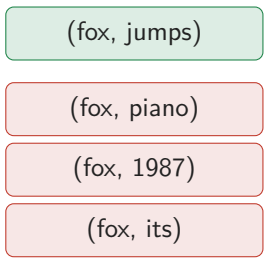
(fox, its)

random words → **fake** (label 0)

usually 5 to 20 of them

Fix #1: negative sampling. Ask “real or fake?”

Replace *“which of 100 000 words is the neighbour?”* with a yes/no question: *“is this pair real?”*



from the text → **real** (label 1)

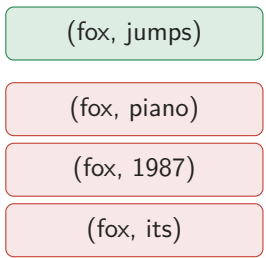
random words → **fake** (label 0)

usually 5 to 20 of them

The model only has to tell real pairs from random ones: **6 dot products instead of 100 000**.

Fix #1: negative sampling. Ask “real or fake?”

Replace *“which of 100 000 words is the neighbour?”* with a yes/no question: *“is this pair real?”*



from the text → **real** (label 1)

random words → **fake** (label 0)
usually 5 to 20 of them

The model only has to tell real pairs from random ones: **6 dot products instead of 100 000**.

The random “fake” words are called *negative samples*.

The learning rule: error = label − prediction

pair	label	model says	error	what happens

The learning rule: error = label − prediction

pair	label	model says	error	what happens
(fox, jumps)	1	0.30	+0.70	pull together, strongly

The learning rule: error = label − prediction

pair	label	model says	error	what happens
(fox, jumps)	1	0.30	+0.70	pull together, strongly
(fox, piano)	0	0.60	−0.60	push apart

The learning rule: error = label − prediction

pair	label	model says	error	what happens
(fox, jumps)	1	0.30	$+0.70$	pull together, strongly
(fox, piano)	0	0.60	-0.60	push apart
(fox, its)	0	0.05	-0.05	almost nothing

The learning rule: error = label − prediction

pair	label	model says	error	what happens
(fox, jumps)	1	0.30	$+0.70$	pull together, strongly
(fox, piano)	0	0.60	-0.60	push apart
(fox, its)	0	0.05	-0.05	almost nothing

Each vector moves a small step in the direction of the other, scaled by the error:

$$\mathbf{u}_o \leftarrow \mathbf{u}_o + \eta \cdot \text{error} \cdot \mathbf{v}_c \qquad \mathbf{v}_c \leftarrow \mathbf{v}_c + \eta \cdot \text{error} \cdot \mathbf{u}_o$$

The learning rule: error = label − prediction

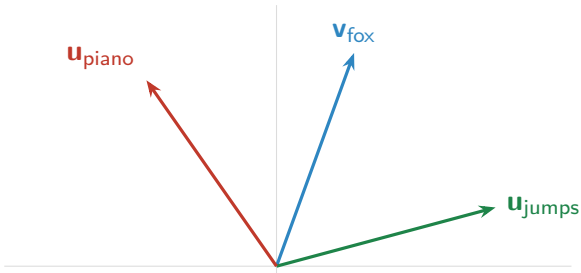
pair	label	model says	error	what happens
(fox, jumps)	1	0.30	$+0.70$	pull together, strongly
(fox, piano)	0	0.60	-0.60	push apart
(fox, its)	0	0.05	-0.05	almost nothing

Each vector moves a small step in the direction of the other, scaled by the error:

$$\mathbf{u}_o \leftarrow \mathbf{u}_o + \eta \cdot \text{error} \cdot \mathbf{v}_c \qquad \mathbf{v}_c \leftarrow \mathbf{v}_c + \eta \cdot \text{error} \cdot \mathbf{u}_o$$

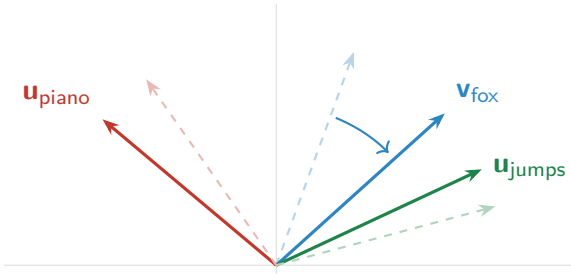
η is the learning rate. Big mistakes $\rightarrow$ big corrections; good guesses $\rightarrow$ tiny ones.

Pull and push, in pictures



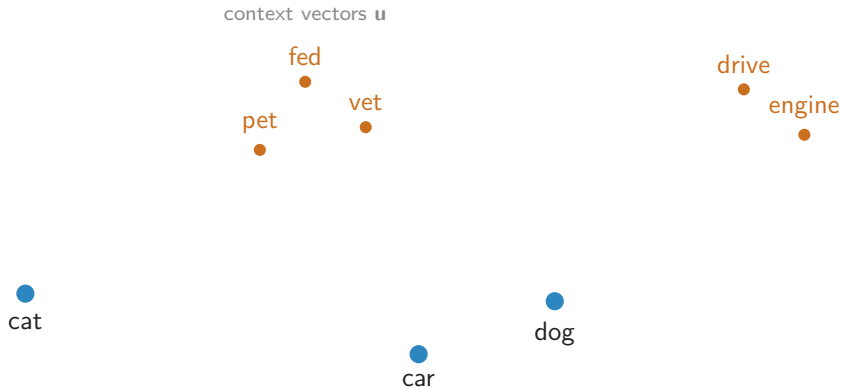
Before the update: *fox* is not especially close to its real neighbour *jumps*.

Pull and push, in pictures



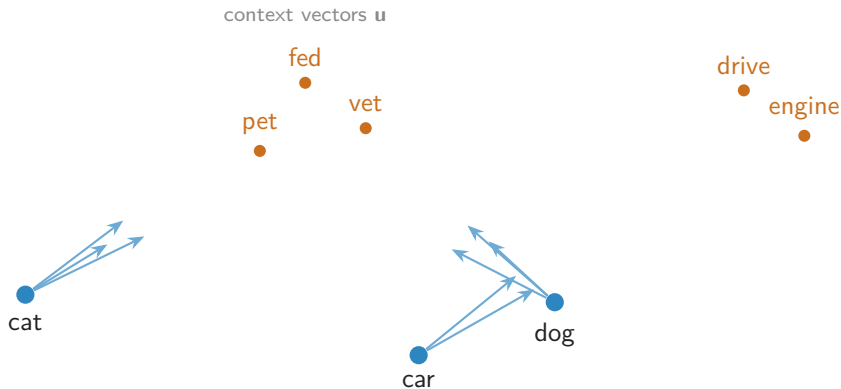
After: *fox* and *jumps* rotate towards each other; *piano* is pushed away.

Why similar words end up close together



cat and *dog* start far apart, at random positions.

Why similar words end up close together



Both keep appearing next to *fed*, *vet*, *pet*, so both get pulled towards the same places.

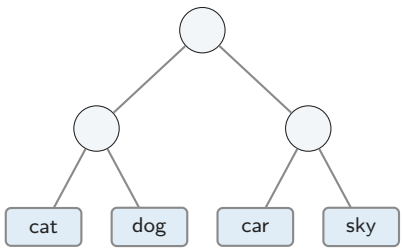
Why similar words end up close together



They end up **close together**. Nobody told the model that cats are like dogs: it follows from **shared contexts**.

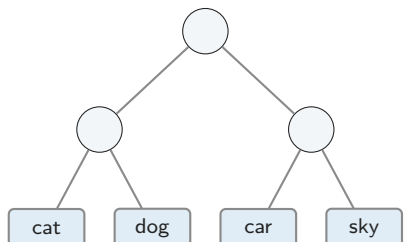
Fix #2: hierarchical softmax. Find the word by asking “left or right?”

- Put all words at the **leaves** of a binary tree.



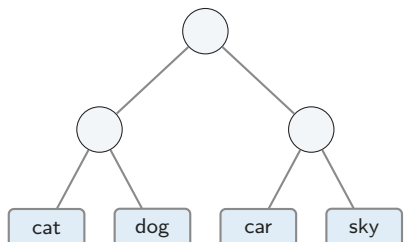
Fix #2: hierarchical softmax. Find the word by asking “left or right?”

- Put all words at the **leaves** of a binary tree.
- Each branching point has its own vector and asks a yes/no question: *go left or right?*



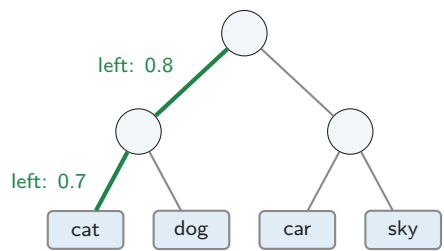
Fix #2: hierarchical softmax. Find the word by asking “left or right?”

- Put all words at the **leaves** of a binary tree.
- Each branching point has its own vector and asks a yes/no question: *go left or right?*
- P(word) = product of the probabilities along its path.



Fix #2: hierarchical softmax. Find the word by asking “left or right?”

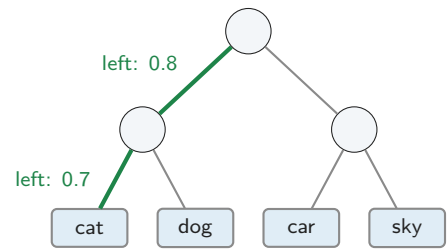
- Put all words at the **leaves** of a binary tree.
- Each branching point has its own vector and asks a yes/no question: *go left or right?*
- P(word) = product of the probabilities along its path.



$P(\text{cat}) = 0.8 \times 0.7 = 0.56$

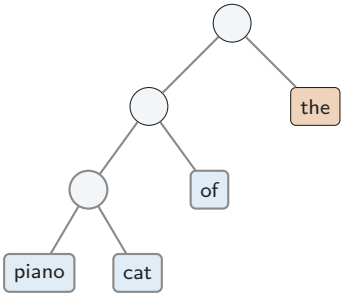
Fix #2: hierarchical softmax. Find the word by asking “left or right?”

- Put all words at the **leaves** of a binary tree.
- Each branching point has its own vector and asks a yes/no question: *go left or right?*
- P(word) = product of the probabilities along its path.
- **100 000 words → about 17 questions** ($\log_2 100\,000$).



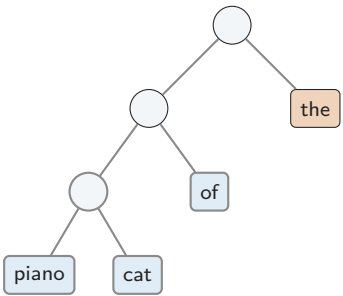
$P(\text{cat}) = 0.8 \times 0.7 = 0.56$

A smarter tree: frequent words get short paths



A **Huffman tree**, as used in file compression

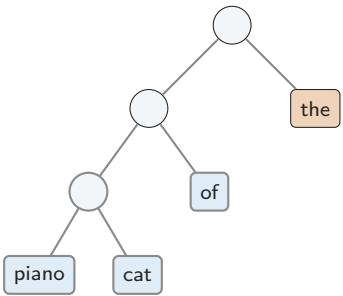
A smarter tree: frequent words get short paths



A **Huffman tree**, as used in file compression

	Negative sampling	**Hierarchical softmax**
Question	real or fake pair?	left or right?
Work per example	5–20 checks	∼17 checks
Tends to be better for	frequent words	rare words

A smarter tree: frequent words get short paths



A **Huffman tree**, as used in file compression

	Negative sampling	**Hierarchical softmax**
Question	real or fake pair?	left or right?
Work per example	5–20 checks	∼17 checks
Tends to be better for	frequent words	rare words

Same idea in both: **replace one enormous choice with a few yes/no choices**.

Two refinements that help a lot

1. Skip some very common words

the makes up about 6% of English text and says little about its neighbours. Randomly **drop most occurrences** of very frequent words:

- training gets much faster
- rare words get relatively more attention

Two refinements that help a lot

1. Skip some very common words

the makes up about 6% of English text and says little about its neighbours. Randomly **drop most occurrences** of very frequent words:

- training gets much faster
- rare words get relatively more attention

2. Pick fake words fairly

Draw negative samples in proportion to **frequency**$^{3/4}$, not raw frequency. This gives rare words a better chance:

word	share of text	share of fakes
the	90%	82%
of	9%	15%
cat	0.9%	2.6%
piano	0.09%	0.46%

Toy 4-word vocabulary.

PART 4

What you get

Nearest neighbours (real output)

Trained on *text8* (17 million words of Wikipedia), 200-dimensional vectors. Numbers are cosine similarity.

france	
spain	0.64
italy	0.63
french	0.61
germany	0.58
belgium	0.57

Nearest neighbours (real output)

Trained on *text8* (17 million words of Wikipedia), 200-dimensional vectors. Numbers are cosine similarity.

france	
spain	0.64
italy	0.63
french	0.61
germany	0.58
belgium	0.57

cat	
meow	0.60
cats	0.59
feline	0.57
dog	0.56
kitten	0.55

Nearest neighbours (real output)

Trained on *text8* (17 million words of Wikipedia), 200-dimensional vectors. Numbers are cosine similarity.

france	
spain	0.64
italy	0.63
french	0.61
germany	0.58
belgium	0.57

cat	
meow	0.60
cats	0.59
feline	0.57
dog	0.56
kitten	0.55

computer	
computers	0.69
computing	0.65
software	0.57
digital	0.53
technology	0.53

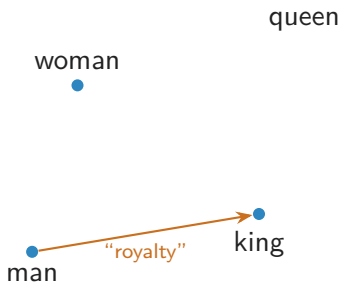
Nearest neighbours (real output)

Trained on *text8* (17 million words of Wikipedia), 200-dimensional vectors. Numbers are cosine similarity.

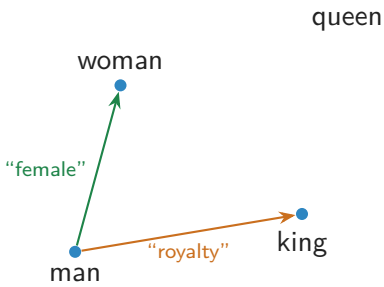
france		cat		computer	
spain	0.64	meow	0.60	computers	0.69
italy	0.63	cats	0.59	computing	0.65
french	0.61	feline	0.57	software	0.57
germany	0.58	dog	0.56	digital	0.53
belgium	0.57	kitten	0.55	technology	0.53

The model was never told any of this. It learned it from which words appear near which.

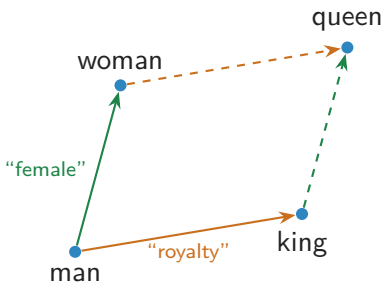
Analogies with vector arithmetic



Analogies with vector arithmetic

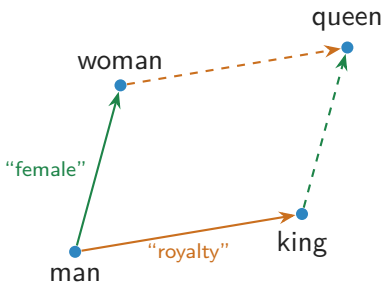


Analogies with vector arithmetic



$$\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} + \mathbf{v}_{\text{woman}} \approx ?$$

Analogies with vector arithmetic

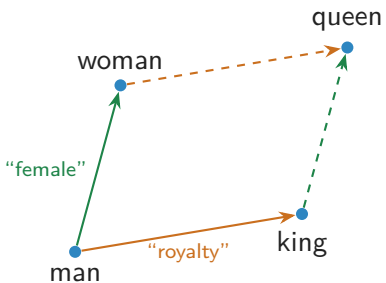


$$\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} + \mathbf{v}_{\text{woman}} \approx ?$$

Real answers (closest word to the result):

king − man + woman	→ **queen** (0.60)
paris − france + germany	→ **berlin** (0.65)

Analogies with vector arithmetic



$$\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} + \mathbf{v}_{\text{woman}} \approx ?$$

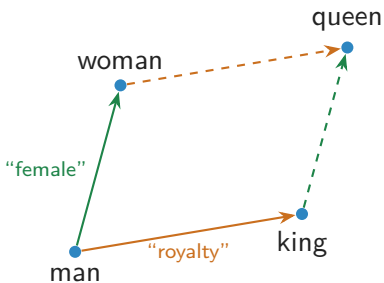
Real answers (closest word to the result):

king − man + woman → **queen** (0.60)
paris − france + germany → **berlin** (0.65)

It doesn't always work:

bigger − big + small → larger (0.66)
then smaller (0.65)

Analogies with vector arithmetic



$$\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} + \mathbf{v}_{\text{woman}} \approx ?$$

Real answers (closest word to the result):

king − man + woman → **queen** (0.60)
paris − france + germany → **berlin** (0.65)

It doesn't always work:

bigger − big + small → larger (0.66)
then smaller (0.65)

Relationships such as gender, capital-of and verb tense become **consistent directions** in the space.

Limitations

- **One vector per word.** *bank* (of a river) and *bank* (money) share one blended vector.

Limitations

- **One vector per word.** *bank* (of a river) and *bank* (money) share one blended vector.
- **Context is ignored at use time.** A word gets the same vector in every sentence.

Limitations

- **One vector per word.** *bank* (of a river) and *bank* (money) share one blended vector.
- **Context is ignored at use time.** A word gets the same vector in every sentence.
- **Unknown words get nothing.** A word never seen in training has no vector, and spelling isn't used.

Limitations

- **One vector per word.** *bank* (of a river) and *bank* (money) share one blended vector.
- **Context is ignored at use time.** A word gets the same vector in every sentence.
- **Unknown words get nothing.** A word never seen in training has no vector, and spelling isn't used.
- **It learns the biases in its text.** E.g. *man : programmer :: woman : homemaker* was found in vectors trained on news (Bolukbasi et al., 2016).

Limitations

- **One vector per word.** *bank* (of a river) and *bank* (money) share one blended vector.
- **Context is ignored at use time.** A word gets the same vector in every sentence.
- **Unknown words get nothing.** A word never seen in training has no vector, and spelling isn't used.
- **It learns the biases in its text.** E.g. *man : programmer :: woman : homemaker* was found in vectors trained on news (Bolukbasi et al., 2016).
- **It needs a lot of text** to produce good vectors.

What came next

2013

word2vec

learn vectors
by predicting
neighbours

What came next

2013

word2vec

learn vectors by predicting neighbours

2014

GloVe

same goal, from global co-occurrence counts

What came next



What came next



What came next



What came next



The core idea of word2vec, **meaning from context, learned by prediction**, is still at the heart of modern language models.

Takeaways

1. **Meaning comes from context**: words with similar neighbours are similar.

Takeaways

1. **Meaning comes from context**: words with similar neighbours are similar.
2. **Invent a prediction game, keep the vectors**: the text provides its own labels.

Takeaways

1. **Meaning comes from context**: words with similar neighbours are similar.
2. **Invent a prediction game, keep the vectors**: the text provides its own labels.
3. **Make it cheap**: negative sampling or hierarchical softmax turn one huge choice into a few yes/no choices.

Takeaways

1. **Meaning comes from context**: words with similar neighbours are similar.
2. **Invent a prediction game, keep the vectors**: the text provides its own labels.
3. **Make it cheap**: negative sampling or hierarchical softmax turn one huge choice into a few yes/no choices.
4. **Geometry encodes meaning**: distance means similarity, and directions capture relationships.

Questions?

Further reading

- Mikolov, Chen, Corrado, Dean. *Efficient Estimation of Word Representations in Vector Space*, 2013.
- Mikolov, Sutskever, Chen, Corrado, Dean. *Distributed Representations of Words and Phrases and their Compositionality*, NeurIPS 2013.
- Goldberg, Levy. *word2vec Explained*, arXiv:1402.3722, 2014.
- Jay Alammar. *The Illustrated Word2vec* (blog post), 2019.
- Bolukbasi et al. *Man is to Computer Programmer as Woman is to Homemaker?*, NeurIPS 2016.